Diseño y Desarrollo de Sistemas de Información

Preparación del entorno para la realización de las prácticas

Curso 2026-27

Software y herramientas

- Apache NetBeans IDE (versión 30)
- DBeaver Community (*Free Universal Database Tool*). Usaremos esta herramienta libre y de código abierto para gestionar la base de datos
- Java Development Kit (JDK) (versión 26)



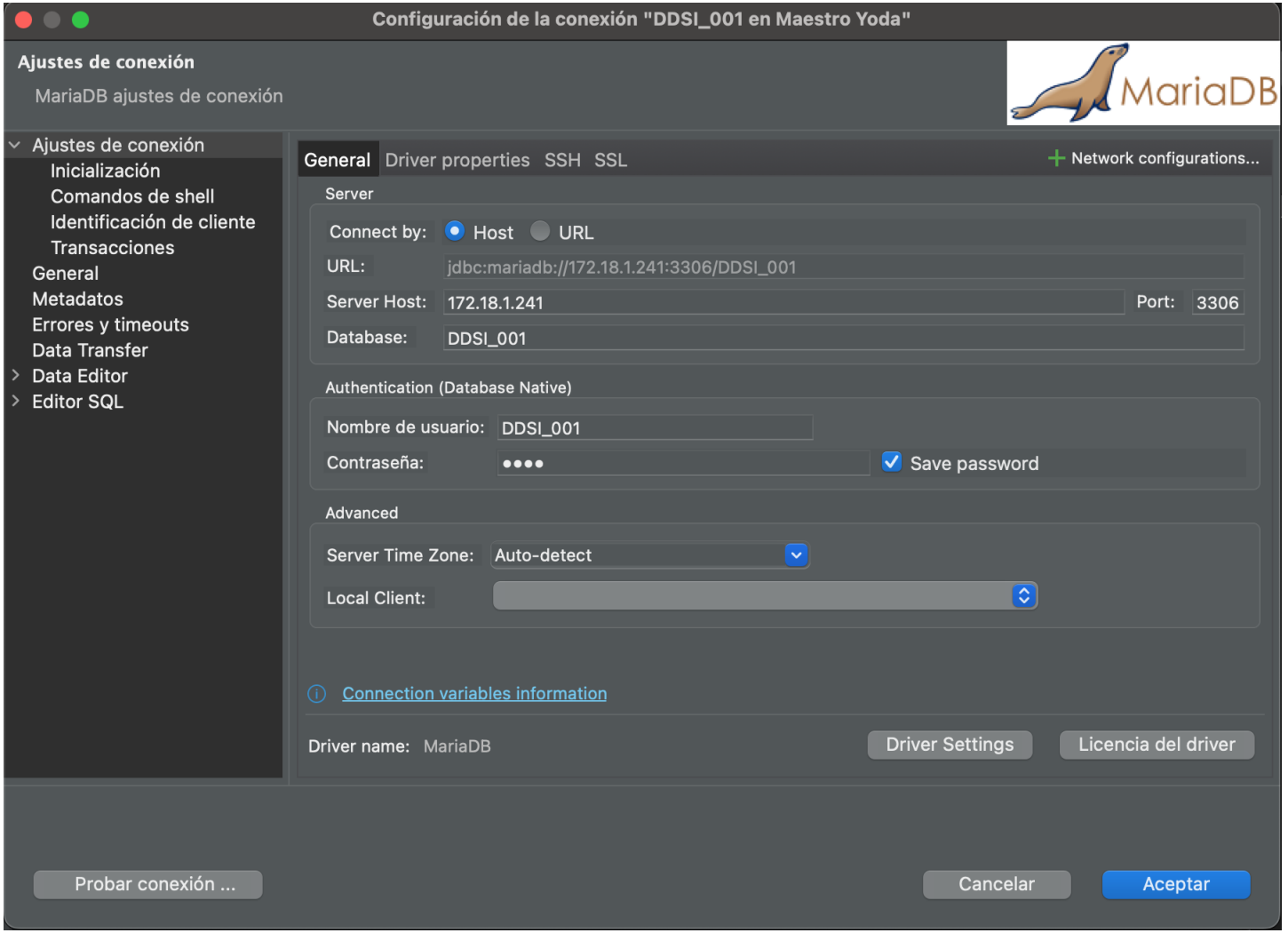




Sistema de Gestión de Base de Datos

- MariaDB

Información de conexión desde DBeaver



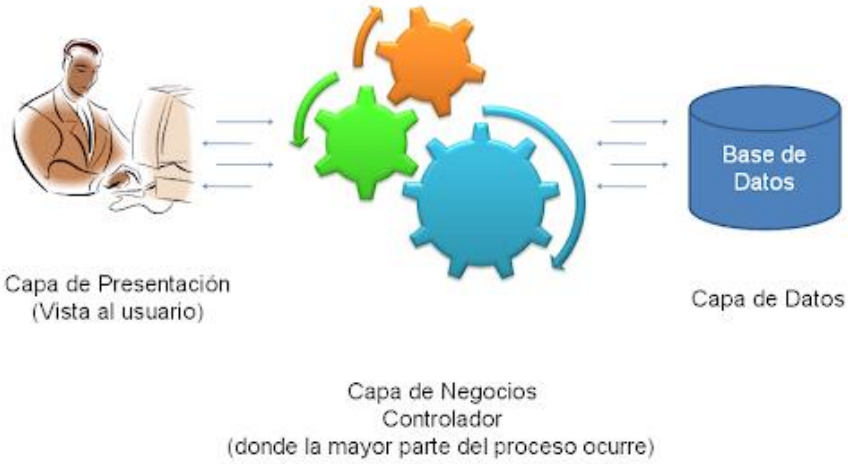
Arquitectura utilizada para el desarrollo del proyecto

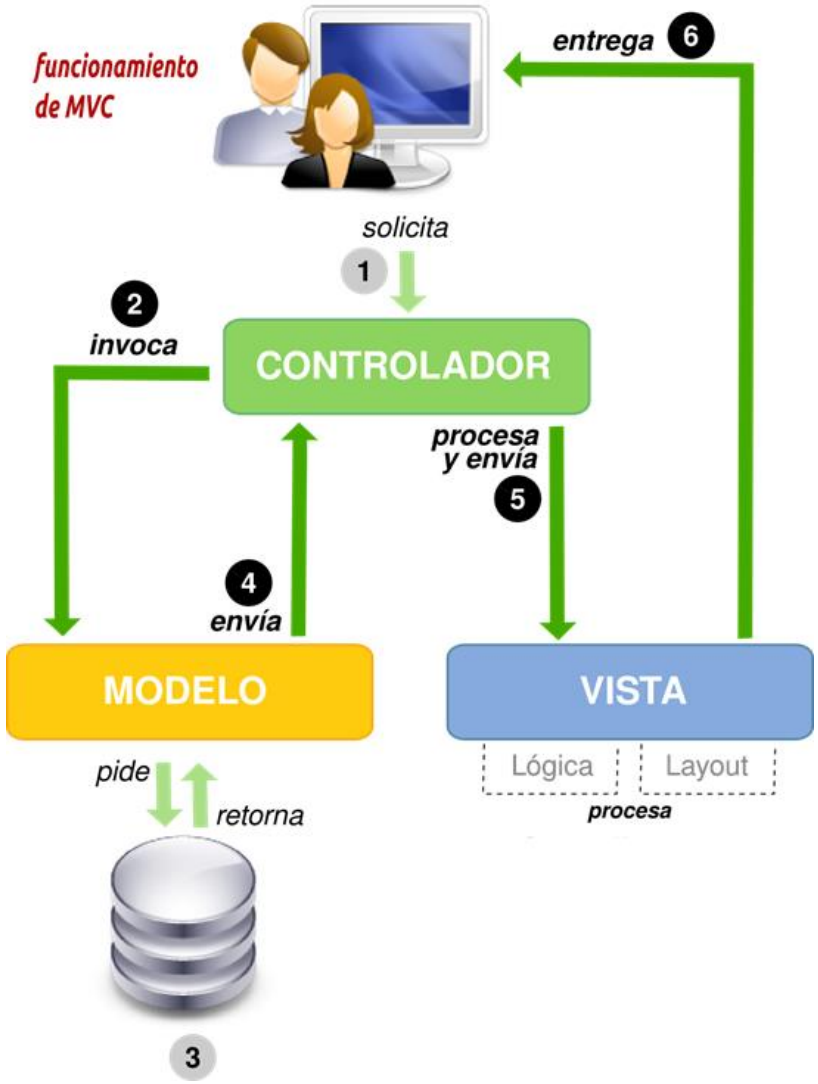
Modelo - Vista - Controlador

- Modelo-Vista-Controlador (MVC) es un patrón de arquitectura de software que que organiza el código según sus responsabilidades, dividiéndolo en capas con tareas específicas

- Se utiliza principalmente en aplicaciones con interfaces de usuario, aunque puede aplicarse a cualquier tipo de aplicación

- Surge de la necesidad de crear software robusto, potenciando la facilidad de mantenimiento, reutilización del código y separación de conceptos


Base de Datos
Capa de Presentación
(Vista al usuario)
Capa de Negocios
Controlador
(donde la mayor parte del proceso ocurre)
Capa de Datos


funcionamiento de MVC
entrega 6
solicita
1
2
invoca
CONTROLADOR
procesa y envía
5
4
envía
MODELO
VISTA
Lógica
Layout
procesa
pide
retorna
3

- **Modelo**
  - Capa encargada de la gestión de los datos de la aplicación
  - Incluye la lógica para acceder, consultar y actualizar la información (generalmente en una base de datos)

- **Vista**
  - Capa responsable de la presentación de la información al usuario
  - Define la interfaz gráfica y cómo se muestran los datos del modelo

- **Controlador**
  - Capa que actúa como enlace entre la Vista y el Modelo
  - Gestiona las acciones del usuario, coordina la lógica y decide qué datos del modelo mostrar en la vista

Crear un proyecto ***Maven*** en *Apache NetBeans* "de un vistazo"

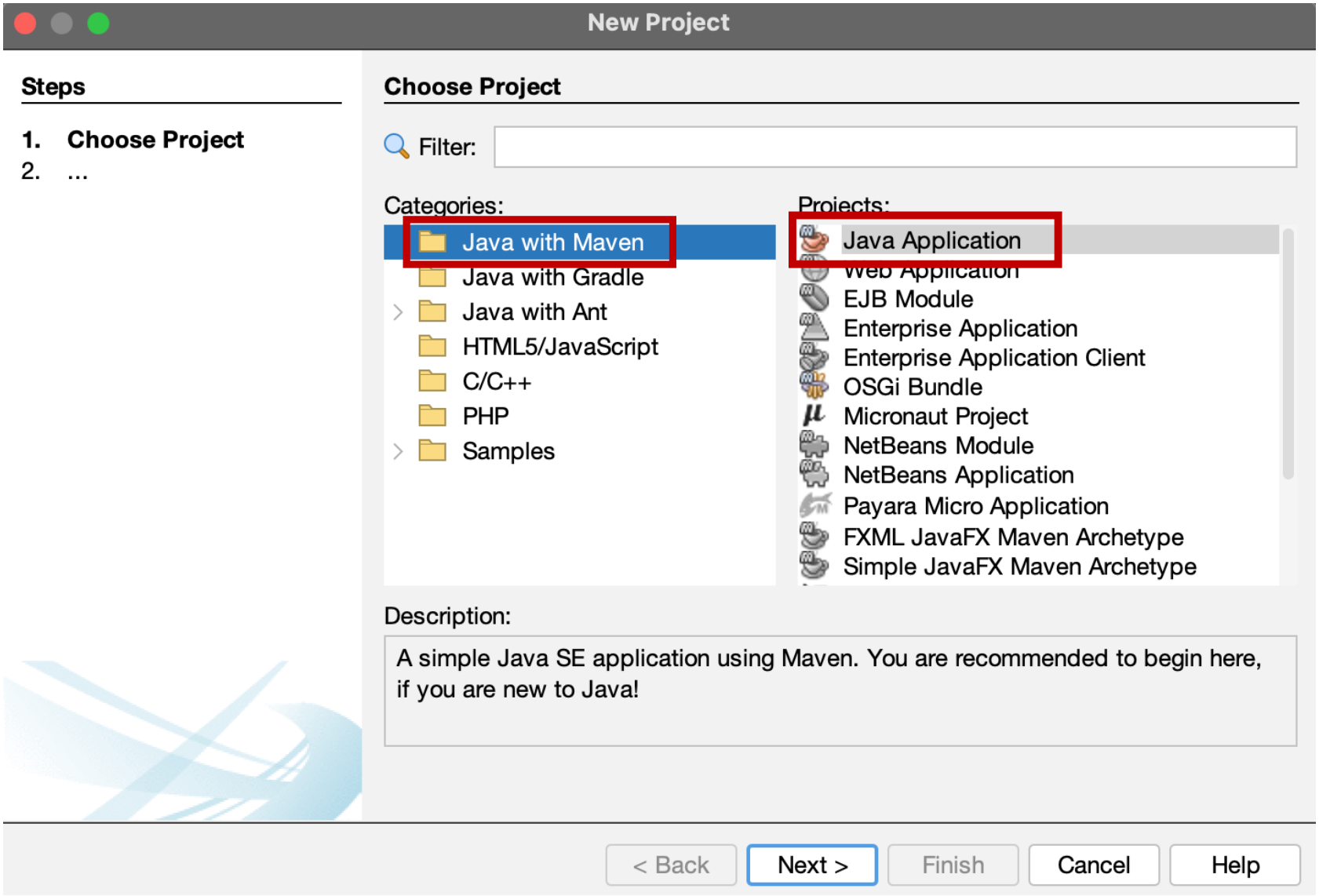
¿Qué es *Maven*?

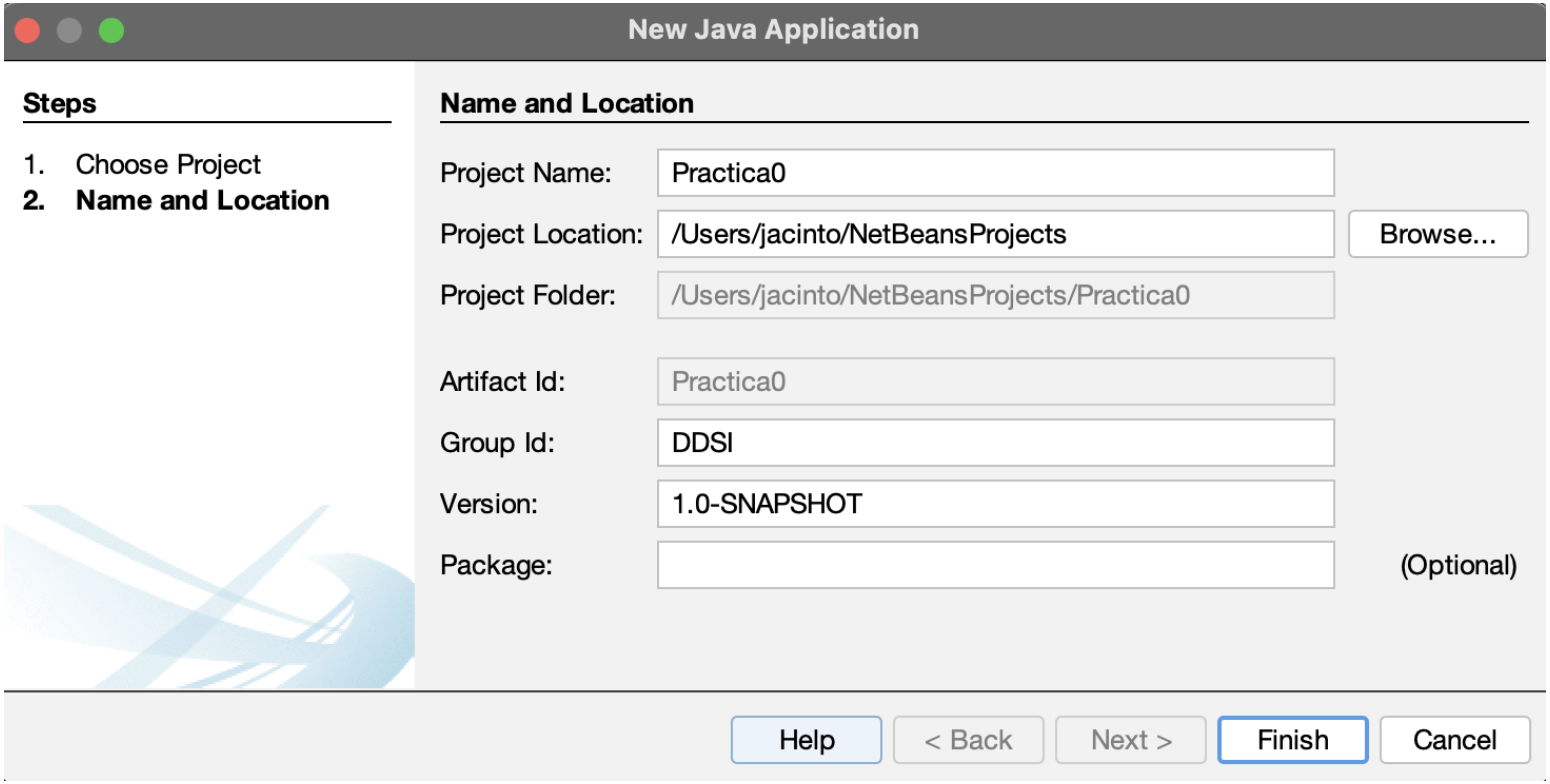
- Herramienta de gestión y construcción de proyectos Java
- *Open source* y de uso gratuito
- Facilita la compilación, empaquetado, gestión de dependencias y despliegue de aplicaciones

Para conocer más sobre el proyecto *Maven*

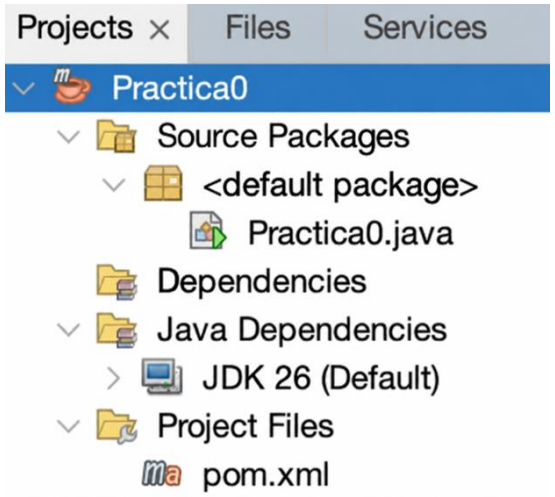
- Página oficial: https://maven.apache.org/
- Maven Central Repository: https://central.sonatype.com/




New Project
Steps
1. Choose Project
2. ...
Choose Project
Filter:
Categories:
Java with Maven
Java with Gradle
Java with Ant
HTML5/JavaScript
C/C++
PHP
Samples
Projects:
Java Application
Web Application
EJB Module
Enterprise Application
Enterprise Application Client
OSGi Bundle
Micronaut Project
NetBeans Module
NetBeans Application
Payara Micro Application
FXML JavaFX Maven Archetype
Simple JavaFX Maven Archetype
Description:
A simple Java SE application using Maven. You are recommended to begin here, if you are new to Java!
< Back
Next >
Finish
Cancel
Help



Estructura original de un proyecto

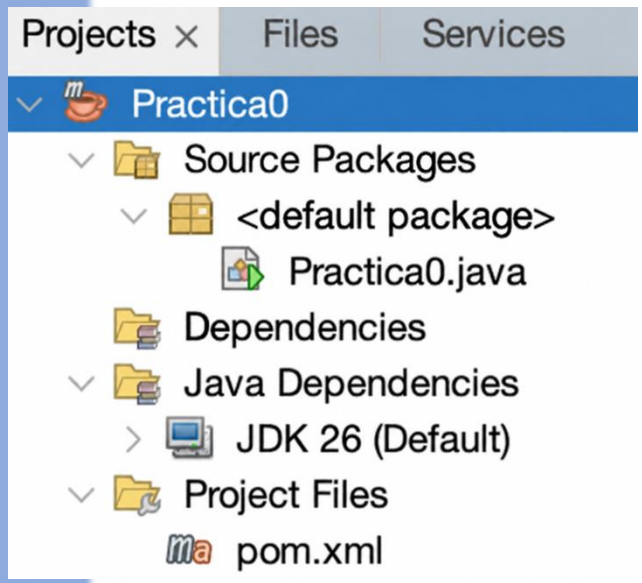


- POM (*Project Object Model*)
  - Archivo principal de configuración de Maven (*pom.xml*)
  - Contiene la información y los parámetros que Maven utiliza para construir y gestionar el proyecto

  - Elementos clave:
    - <artifactId> → Nombre único del proyecto dentro de un grupo
    - <groupId> → Identifica al grupo o conjunto de proyectos relacionados
    - <version> → Estado del proyecto (ej. SNAPSHOT para desarrollo, sin SNAPSHOT para producción)
    - <maven.compiler.release> → Indica la versión de Java para la cual se compilará el código.

- En este curso lo usaremos, principalmente, para añadir las dependencias (por ejemplo, los conectores para acceder al SGBD).

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>DDSI</groupId>
    <artifactId>Practica0</artifactId>
    <version>1.0-SNAPSHOT</version>
    <packaging>jar</packaging>
    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.release>26</maven.compiler.release>
        <exec.mainClass>Practica0</exec.mainClass>
    </properties>
</project>
```

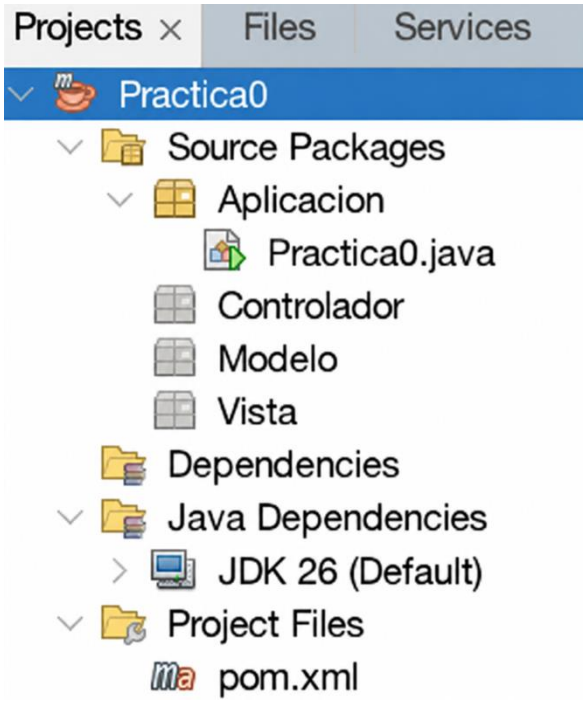


En la carpeta "Source Packages" crearemos la estructura de paquetes (capas) que tendrá el proyecto

Con el botón derecho en "Source Package" seleccionamos "New" y "Java Package" y creamos los paquetes Modelo, Vista, Controlador y Aplicacion **(sin tilde)**

El fichero .java que contiene la clase principal **main()**, y que se genera de forma automática, se trasladará al paquete "Aplicacion"

Finalmente, la estructura general del proyecto se muestra en esta figura



En las propiedades del proyecto (botón derecho) hay que especificar cuál es el fichero que contiene el método ***main()***. Dentro de ***"Run"***, en la propiedad ***"Main Class"***

- Para incluir las distintas dependencias que vaya necesitando nuestro proyecto, editamos el fichero pom.xml y añadimos la sección <dependencies> </dependencies>

```
<dependencies>
    <dependency>
        <groupId>org.hibernate.orm</groupId>
        <artifactId>hibernate-core</artifactId>
        <version>7.4.7.Final</version>
    </dependency>

    <dependency>
        <groupId>jakarta.persistence</groupId>
        <artifactId>jakarta.persistence-api</artifactId>
        <version>3.2.0</version>
    </dependency>

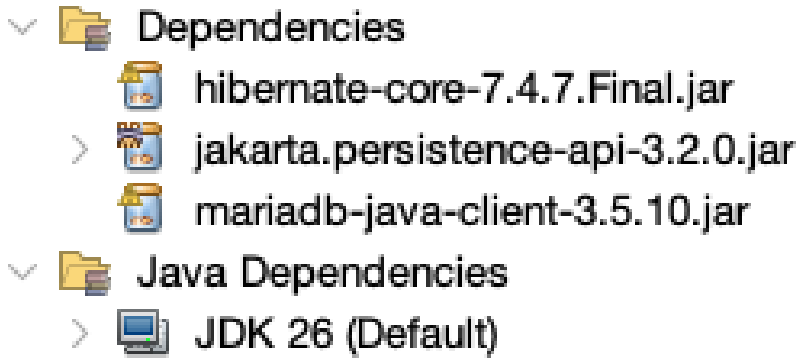
    <dependency>
        <groupId>org.mariadb.jdbc</groupId>
        <artifactId>mariadb-java-client</artifactId>
        <version>3.5.10</version>
    </dependency>
</dependencies>
```

→ Dependencia para trabajar con Hibernate 7

→ Dependencia para trabajar con la persistencia de Java

→ Dependencia para la conexión con el servidor MariaDB

- Al grabar el fichero pom.xml, en la carpeta **"Dependencies"** aparecerán las librerías **".jar"** necesarias para desarrollar el proyecto



Con el botón derecho en el proyecto, seleccionamos **"Build with Dependencies"** para descargarlas y enlazarlas con el programa